# Step-by-Step Machine Learning Workflow

YojanaMitra — government scheme eligibility as a binary classification problem

Lab report · the eleven-step workflow from the handout, applied to the YojanaMitra project's own benchmark data · all results reproduced from the accompanying executed notebook `YojanaMitra_ML_Workflow.ipynb`

| 20 | 40 | 800 | 106 | 19% | 0.67 |
|---|---|---|---|---|---|
| PERSONAS | SCHEMES | LABELLED PAIRS | FEATURES | ELIGIBLE | BEST F1 |

> **What this experiment asks.** YojanaMitra decides eligibility with a *deterministic rule engine*: every scheme carries machine-readable clauses ( `annual_income_inr <= 700000` , `occupation in [farmer]` ) and a citizen passes or fails each one. The benchmark labels are the output of running that engine over hand-written ground-truth profiles. So the question here is precise and worth answering: **can standard baseline ML models recover those eligibility rules from labelled examples alone, never having been shown the rules?** That decides whether the hand-written engine could be replaced by a learned model. It is not a claim that the scheme criteria are legally correct — the corpus is a development seed and its own provenance note says so.

## 1. Dataset Understanding

| | |
|---|---|
| **Dataset name** | YojanaMitra scheme-eligibility benchmark |
| **Source** | The project's own `data/` directory — `schemes.json` (the scheme corpus with eligibility clauses) and `gold_set.json` (the labelled persona benchmark) |
| **Unit of analysis** | One row = one *(citizen persona, scheme)* pair |
| **Number of samples** | 20 distinct personas × 40 schemes = **800** labelled pairs (from 60 benchmark utterances — see step 5) |
| **Input features** | 106 engineered features across three groups: *citizen* (age, income, land, residence, social category, occupation, special categories, education stage, dependents), *scheme* (identity, central/state, clause count, scheme category) and *interaction* ( `state_match` ) |
| **Target variable** | `eligible` — 1 if the scheme appears in that persona's `expected_schemes` , else 0 |
| **Problem type** | **Classification** — the target is a discrete label |
| **Number of classes** | 2 (binary): eligible / not eligible, split 19.1% / 80.9% |

The benchmark ships two label sets: `expected_schemes` , the strict rule-engine answer, and `acceptable_schemes` , a wider tolerance band. The strict set is used throughout, because a fuzzy target would make every metric in step 9 impossible to interpret.

## 2. Load the Dataset

The two source files are nested JSON, not a table, so this step is a genuine transformation rather than a one-line read. Every persona is crossed with every scheme, and each resulting pair is flattened into a feature row: numeric citizen attributes, multi-hot blocks for occupation, special categories, education stage and social category, scheme metadata, and one interaction feature — `state_match`, which encodes that a state scheme only applies inside its own state. Result: 60 × 40 = 2400 raw rows.

## 3. Explore the Dataset (EDA)

- **Missing values are structural, not random.** `land_acres` is absent whenever the persona is not a landholder at all — a student, a street vendor, a weaver. The absence is itself informative, which changes how it should be handled in step 5.
- **The benchmark triple-counts every persona.** Each of the 20 personas appears three times, as a *plain*, *telegraphic* and *code-mixed* phrasing of the same underlying profile. Different NLP inputs, identical structured features. This is the single most consequential finding in the exploration.
- **The target is imbalanced** at roughly one eligible pair in five. Most citizens qualify for a minority of the 40 schemes.
- **No single feature predicts the target.** Every correlation with `eligible` is weak, because eligibility is a *conjunction* of clauses — income *and* occupation *and* state *and* age. This is the signature of a rule-based target and it predicts, before any model is trained, that linear models will struggle where tree models will not.
- **4 of the 40 schemes are matched by no persona at all** and contribute only negative examples — a real coverage limitation of the benchmark.

## 4. Visualise the Data

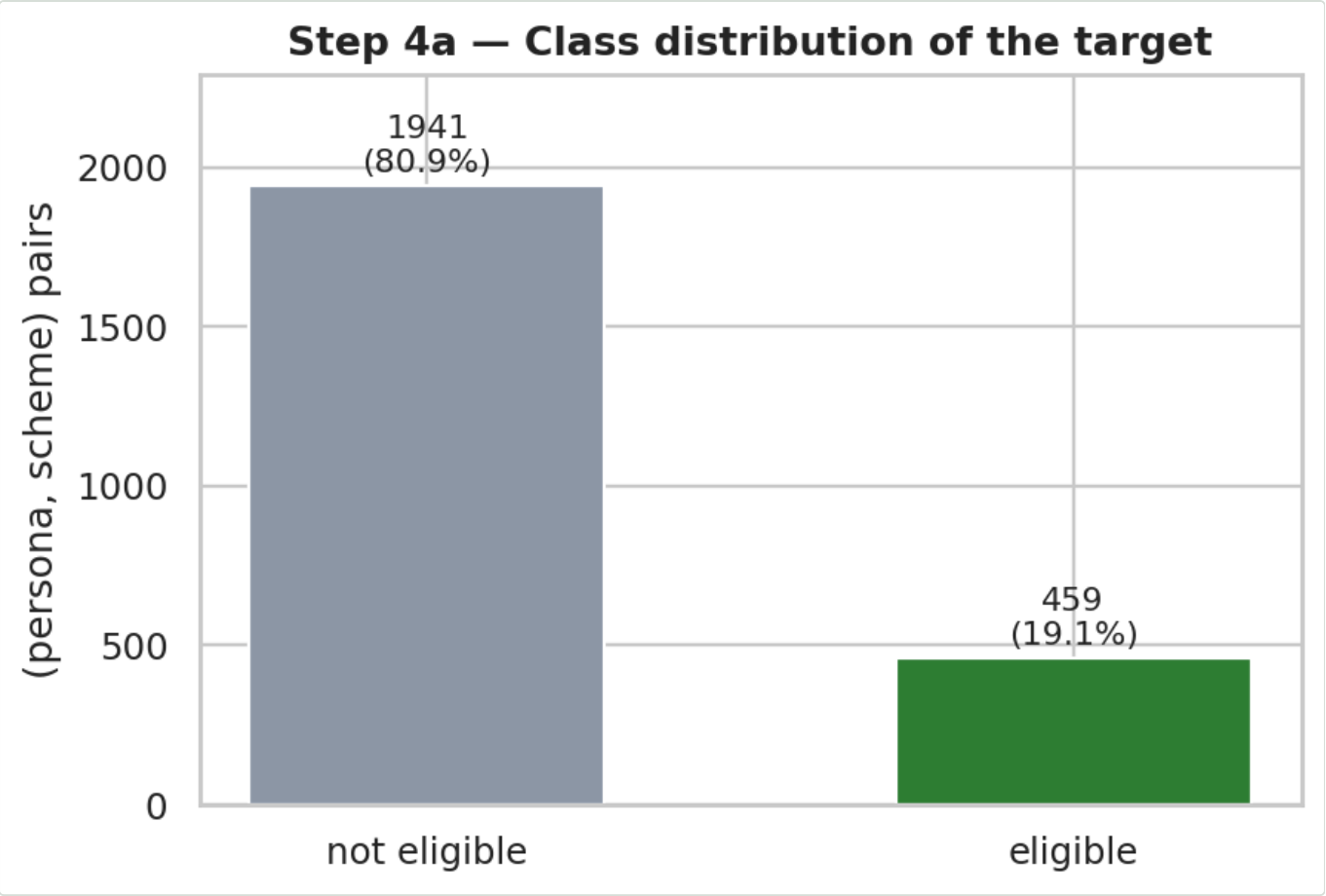



*Figure 1 (bar chart) — class distribution of the target. Roughly one pair in five is a genuine entitlement.*

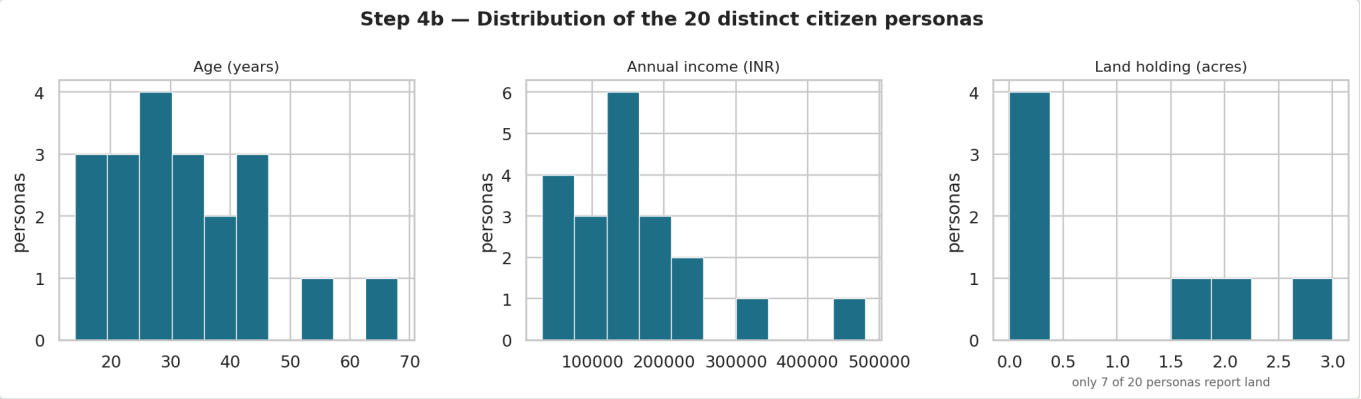



*Figure 2 (histogram) — the 20 distinct citizen personas by age, income and land. Note how few report land at all.*

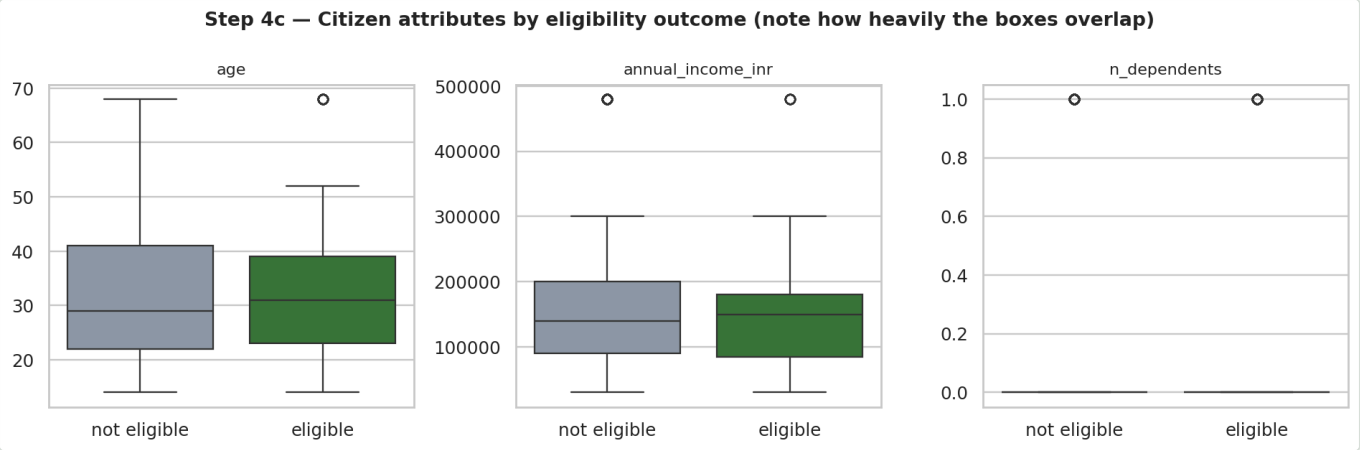



*Figure 3 (box plot) — citizen attributes split by eligibility outcome. The boxes overlap almost completely: no single attribute decides the answer.*

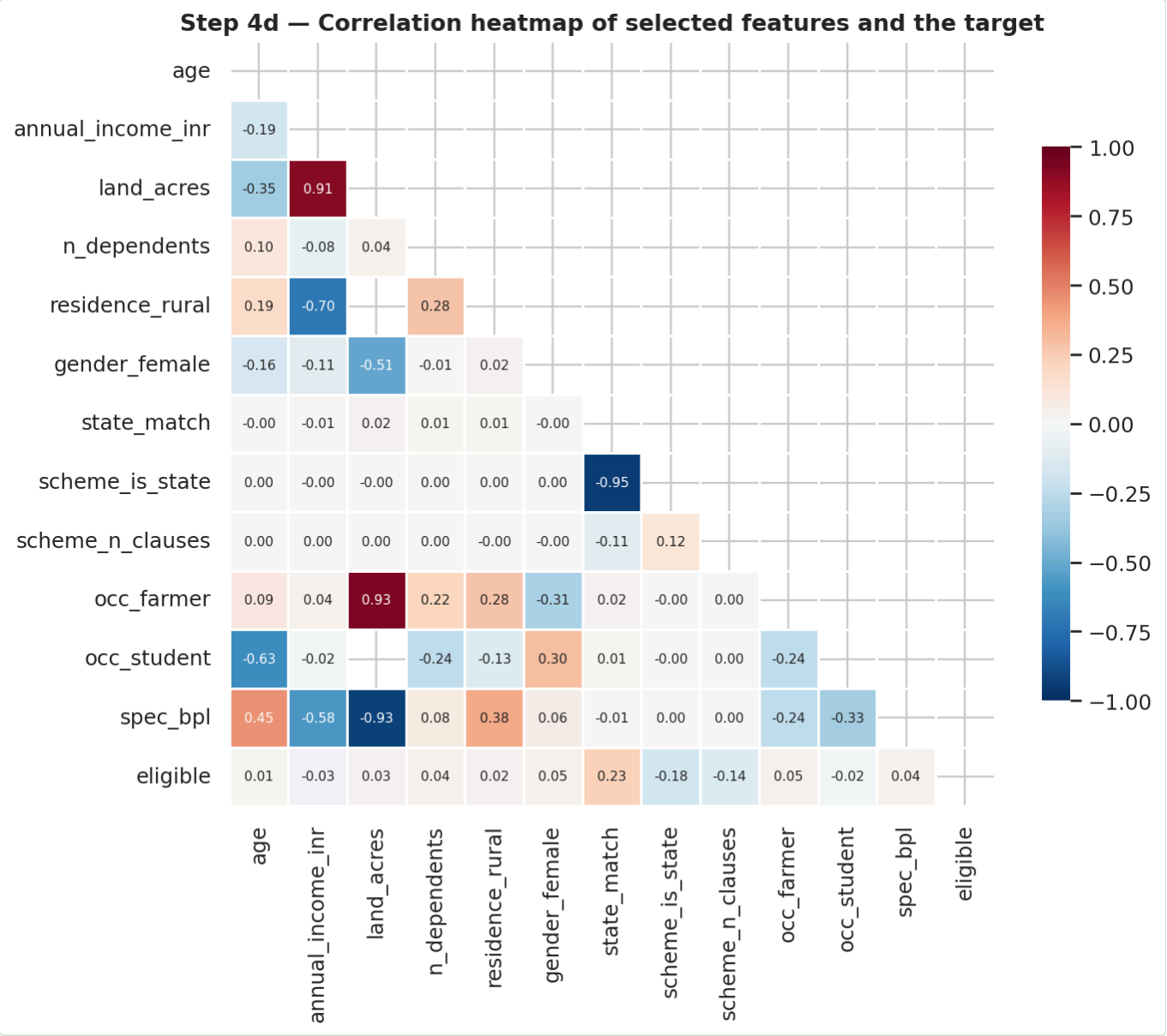



*Figure 4 (correlation heatmap) — the 'eligible' row is almost entirely pale. That is the most important thing this figure says.*

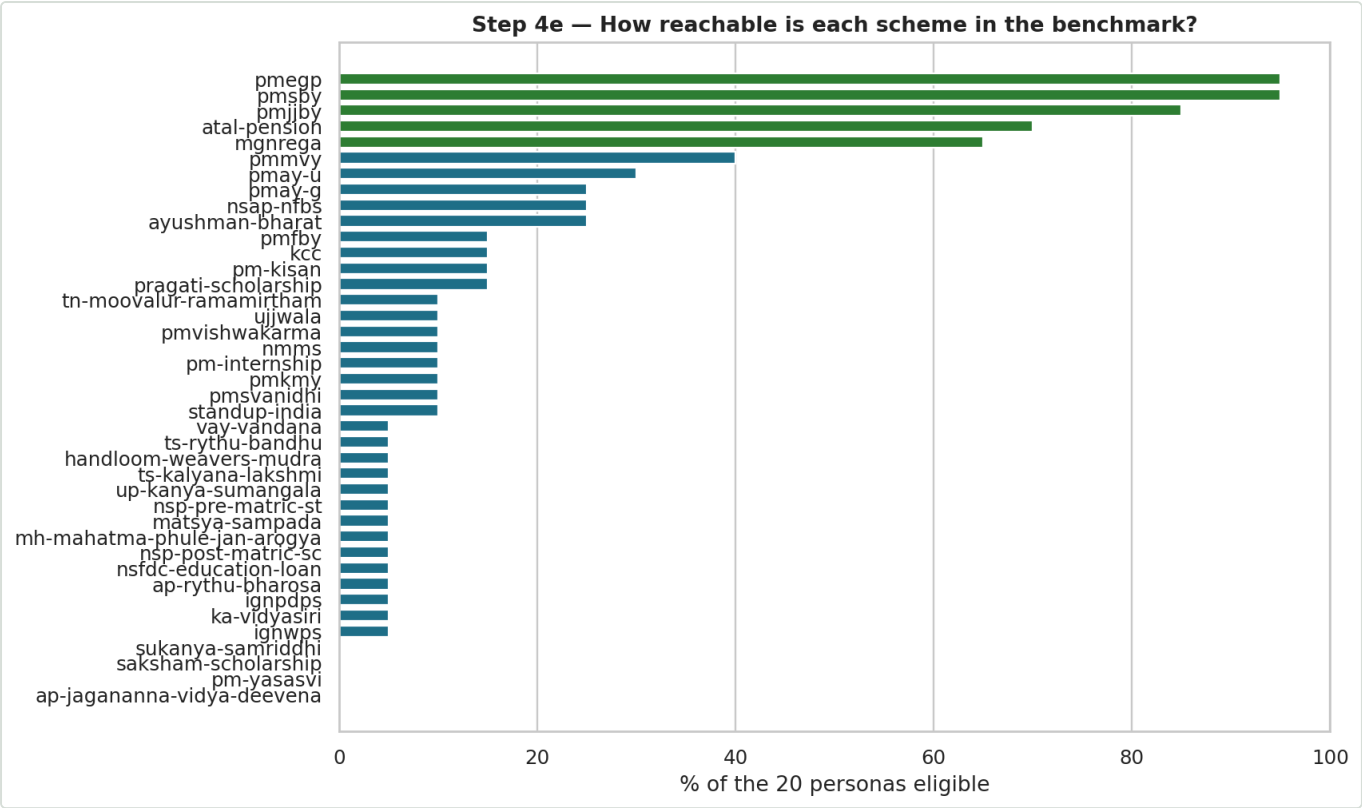



*Figure 5 — how reachable each scheme is across the 20 personas. Red bars are schemes no persona in the benchmark matches.*

## 5. Clean and Preprocess the Data

| | |
|---|---|
| **De-duplication** | 2400 → **800** rows — one utterance kept per persona |
| **Missing values** | Imputed *with an explicit indicator column*, not a silent fill |
| **Zero-variance features** | Checked and dropped where found |
| **Encoding** | `scheme_id` one-hot encoded into 40 columns; all other features already numeric or binary |
| **Scaling** | Attached to each model as a `Pipeline` stage in step 8, not applied here — see the note below |

**On the imputation.** `land_acres` is filled with 0 rather than the column median, because a missing land field means the persona is not a landholder, and 0 acres is the truthful value — the median would invent farmland for a student. A companion `land_acres_missing` flag is added so a model can still distinguish "genuinely landless" from "not stated".

> **Preventing data leakage — a deliberate deviation in ordering.** The handout places preprocessing at step 5 and the split at step 7. Fitting a `StandardScaler` on the full dataset at step 5 would let the test rows influence the training statistics and inflate every score reported later. Dataset-level cleaning is therefore done here, while **scaling is attached to each model as a `Pipeline`** in step 8, so the scaler is fitted on the training fold alone. Same operations, correct order, honest numbers.

## 6. Separate Input and Output

`X` holds the 106 engineered features; `y` holds the eligibility label. A third array, `groups`, records which persona each row came from. It is deliberately *not* a feature — but step 7 cannot be done correctly without it.

## 7. Split the Dataset

This is where this dataset departs from a textbook one. Each persona contributes 40 rows, one per scheme. A plain random split scatters those rows across train and test, so the model sees *the same citizen* during training and is then asked about that citizen again at test time. It only has to remember, not generalise.

The correct split is **grouped by persona**: a persona is entirely in train or entirely in test, never both. Both splits were run so the difference could be measured rather than asserted.

> **The leakage, measured.** Identical model (Random Forest, 300 trees), identical data, two split strategies:
>
> | | |
> |---|---|
> | **Random split (leaky)** | accuracy 0.9417 · **F1 0.8372** |
> | **Grouped split (honest)** | accuracy 0.8625 · **F1 0.5926** |
>
> A careless split would have reported an F1 **0.245 higher** than the model actually achieves on citizens it has never seen. Everything that follows uses the grouped split: **560 training rows from 14 personas**, **240 test rows from 6 completely unseen personas**.

## 8. Train Baseline ML Models

Six baselines spanning the main model families, plus a trivial reference point. The *Majority Baseline* always predicts "not eligible"; it is included precisely because it exposes how badly accuracy flatters a model on 19%-positive data.

| | |
|---|---|
| **Majority Baseline** | Trivial reference — always predicts the majority class |
| **Logistic Regression** | Linear, probabilistic — scaled |
| **K-Nearest Neighbours (k=5)** | Distance-based — scaled |
| **Decision Tree** | Single tree, axis-aligned splits — unscaled |
| **Random Forest (300 trees)** | Bagged tree ensemble — unscaled |
| **Support Vector Machine (RBF)** | Margin-based, kernelised — scaled |
| **Gaussian Naive Bayes** | Probabilistic, independence assumption — unscaled |

## 9. Evaluate Each Model

For YojanaMitra the two errors are not symmetric. A **false negative** — failing to surface a scheme the citizen actually qualifies for — means a citizen never learns about a benefit they are entitled to, which is the failure the whole project exists to prevent. A **false positive** wastes an application and erodes trust, but is recoverable. So **recall on the eligible class** is the headline number and F1 the balanced view; accuracy is reported only to show how misleading it is here. Cross-validation uses `StratifiedGroupKFold`, keeping personas intact in every fold.

| # | Model | Accuracy | Precision | Recall | F1 | ROC-AUC | CV F1 (mean) |
|---|---|---|---|---|---|---|---|
| 1 | **Decision Tree** | **0.8750** | 0.7143 | **0.6250** | **0.6667** | 0.7812 | **0.8450** |
| 2 | **Random Forest** | 0.8625 | **0.7273** | 0.5000 | 0.5926 | 0.8326 | 0.7923 |
| 3 | **Logistic Regression** | 0.8542 | 0.7097 | 0.4583 | 0.5570 | 0.7815 | 0.6599 |
| 4 | **SVM (RBF kernel)** | 0.8500 | 0.7000 | 0.4375 | 0.5385 | **0.8504** | 0.6968 |
| 5 | **K-Nearest Neighbours** | 0.8500 | 0.7000 | 0.4375 | 0.5385 | 0.8080 | 0.7229 |
| 6 | ***Majority Baseline*** | *0.8000* | ***0.0000*** | ***0.0000*** | ***0.0000*** | *0.5000* | ***0.0000*** |
| 7 | **Gaussian Naive Bayes** | 0.8000 | **0.0000** | **0.0000** | **0.0000** | 0.5689 | **0.0000** |

Precision, Recall and F1 are all for the *eligible* class. Green cells mark the best value among the real models; red zeros mark models that never predict "eligible" at all. The Majority Baseline row is shaded because it is a reference point, not a competitor.

> **Read the bottom two rows together.** The Majority Baseline scores 80% accuracy while recommending *nothing to anybody* — recall 0.0000. Gaussian Naive Bayes scores exactly the same, because it too collapses to predicting "not eligible" for every pair. Any report that led with accuracy would have called both of these respectable models.

## 10. Generate the Confusion Matrix

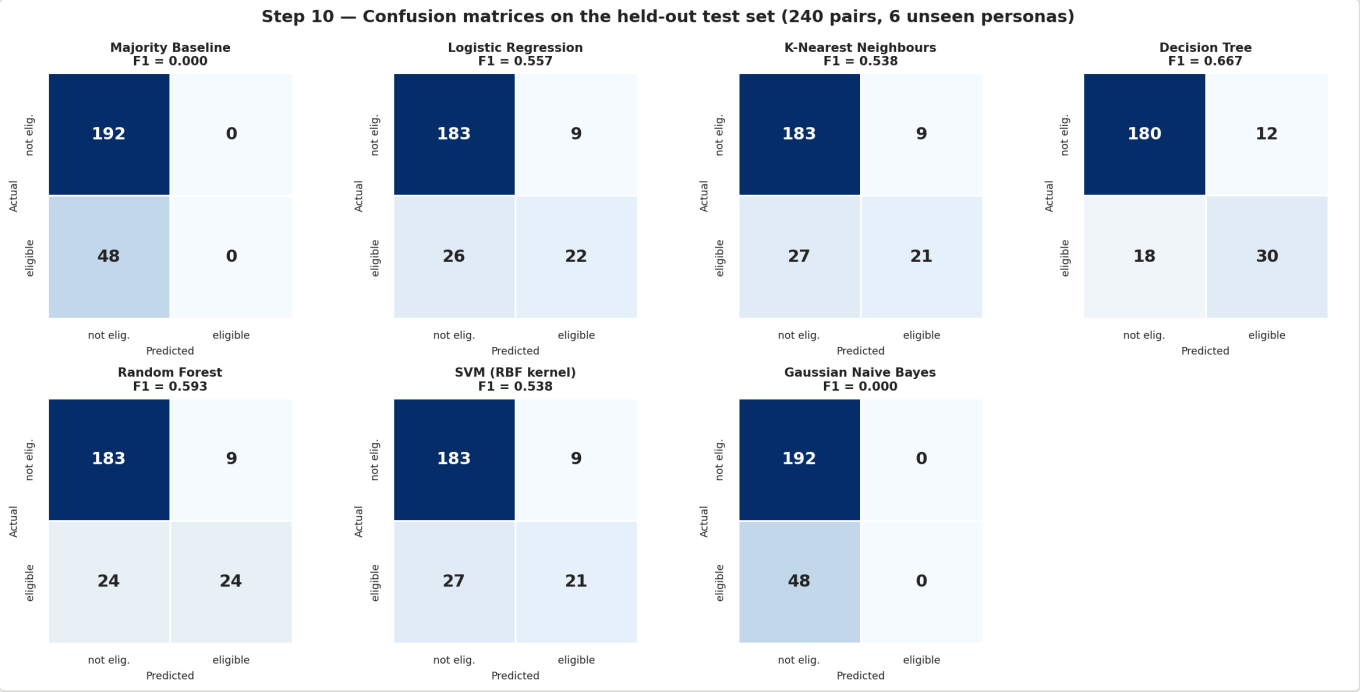



*Figure 6 — confusion matrices on the held-out test set. The bottom-left cell of each is the count of entitlements the model would have failed to surface.*

**Reading the best model (Decision Tree).** Of the 48 genuine entitlements among the 6 unseen personas, it correctly surfaced 30 and **missed 18**. Its precision of 0.714 means roughly 29% of what it does recommend would be a wasted application. Neither number is good enough to put in front of a citizen.

## 11. Compare All Baseline Models

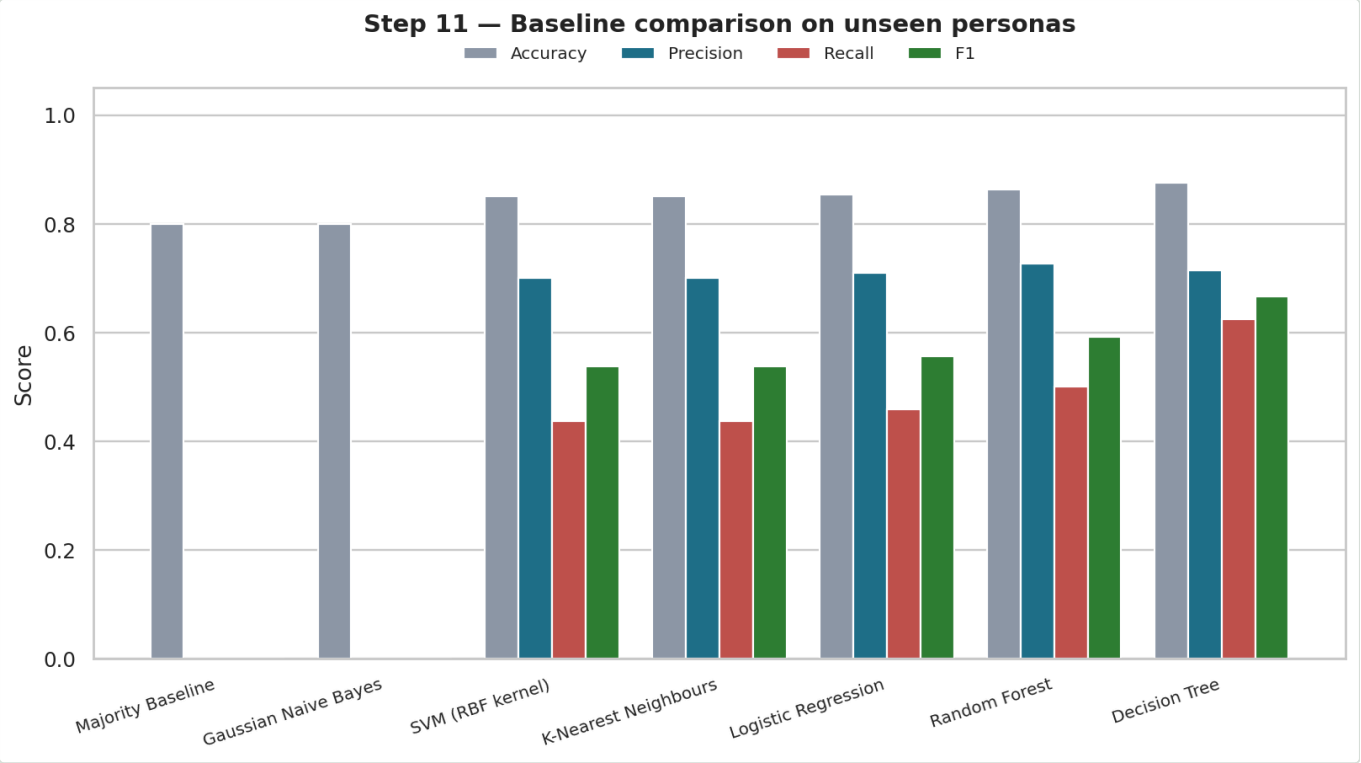



*Figure 7 — all four metrics across every model. The two left-hand groups show tall accuracy bars beside nothing at all: the clearest possible argument against reporting accuracy on imbalanced data.*

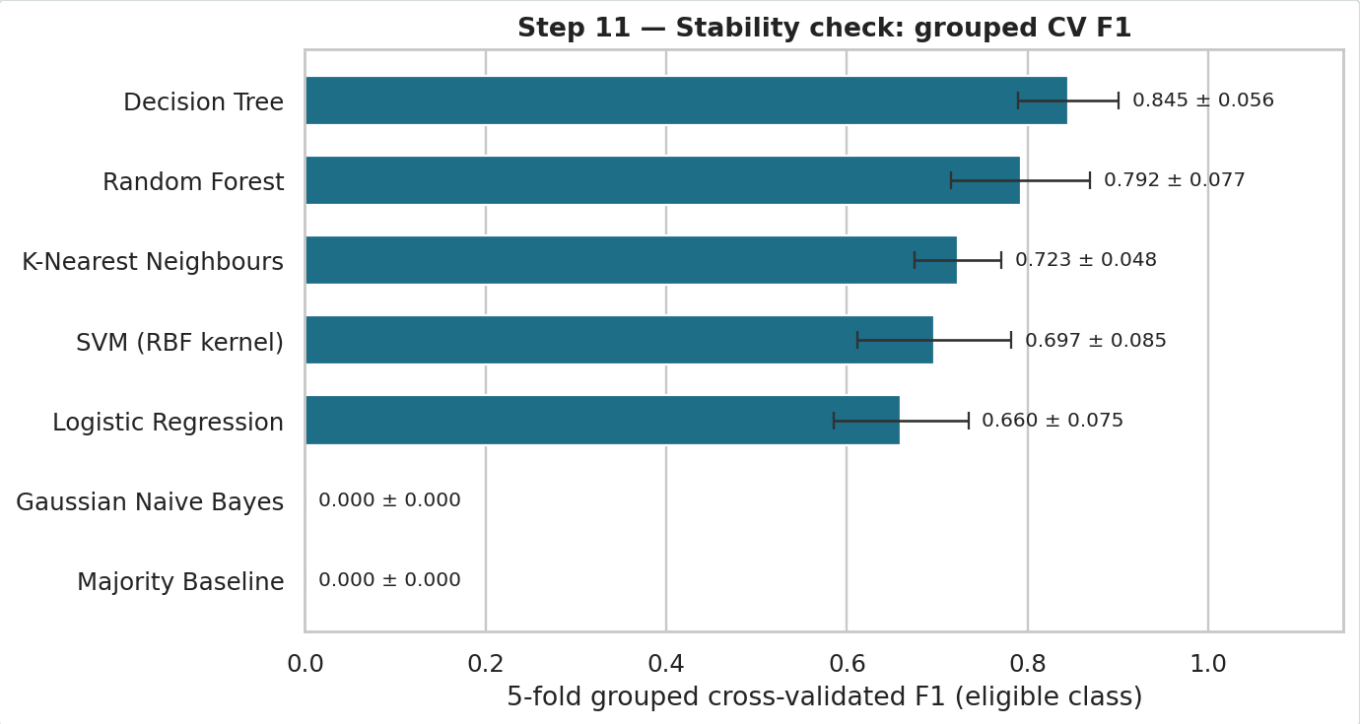



*Figure 8 — grouped 5-fold cross-validated F1 with standard deviation. The error bars are wide because each fold trains on roughly 11 personas.*

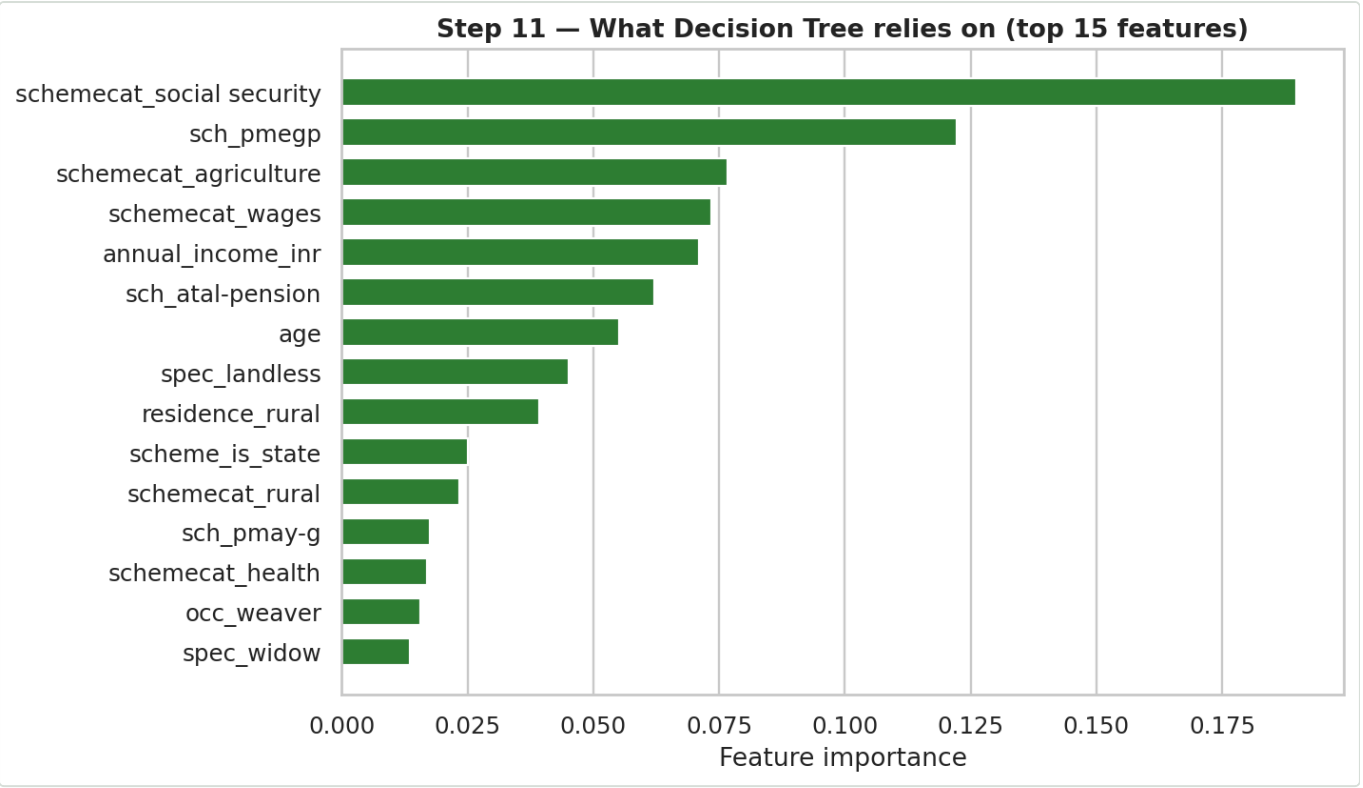



*Figure 9 — what Decision Tree actually relies on. Scheme-side features dominate the top of the chart; citizen attributes appear lower down.*

> **The most revealing result, and not the flattering one.** Summed over all 106 features, Decision Tree places **68% of its importance on scheme-side features** (which scheme is this, what category is it) and only **32% on citizen-side features**. But the corpus constrains `annual_income_inr` in 15 clauses, `age` in 15, `special_categories` in 11, `occupation` in 10 and `education_stage` in 10 — so a model that had genuinely learned the *rules* would lean on those citizen fields. Instead the tree has largely learned each scheme's *base rate*, how often it is granted across the training personas, and uses income and age only as a secondary correction. That is memorisation of the label distribution, not recovery of the eligibility logic.

### What this means for YojanaMitra

- **Tree models lead, as the EDA predicted.** Decision Tree takes the best F1 at 0.6667 and the best cross-validated F1 at 0.8450. Eligibility is a conjunction of threshold tests, which is the shape a tree represents natively and a linear boundary cannot.
- **Naive Bayes fails completely**, predicting "not eligible" for every pair. Its independence assumption is fatal here: eligibility depends on features *interacting*, which is exactly what that assumption forbids.
- **The rule engine is not beaten — it defined the labels**, so it scores 1.0 by construction. The honest reading runs the other way: the gap between 0.67 and 1.0 measures how much of the rule set is *not* recoverable from 14 training personas.
- **Even at equal accuracy, the learned model would be the wrong choice.** YojanaMitra has to tell a citizen *which clause* they failed and why. The rule engine emits that for free; a tree emits a probability. Feature importances are a global summary, not a per-citizen explanation, and an eligibility decision a citizen cannot contest is not a usable one.

- **The bottleneck is data, not algorithms.** 20 personas over 40 schemes is a demonstration, not a training set. Any serious attempt needs hundreds of verified profiles — and the corpus itself needs verifying against the live portal before any accuracy claim leaves this notebook.

### Limitations

- 14 training personas is very small. The wide cross-validation error bars in Figure 8 are the honest expression of that, and small differences in the ranking should not be trusted.
- The scheme corpus is a development seed, paraphrased from published criteria and explicitly *not* verified against the live portal. No claim here is a claim about Indian eligibility law.
- Modelling from the structured ground-truth profile bypasses YojanaMitra's actual hard part — extracting that profile from free text. The three phrasings per persona exist to test exactly that, and this experiment deliberately sets it aside.
- All models use default hyperparameters. Tuning, and threshold adjustment to trade precision for the recall this application needs, are the obvious next steps.

---

All figures and numbers in this report were generated by the accompanying notebook `YojanaMitra_ML_Workflow.ipynb` and read back from `model_results.csv` and `run_summary.json`, so the two documents cannot disagree. Reproducible with `random_state = 42`. Test personas held out: ap-small-farmer-daughter, disabled-adult-bpl, sc-engineering-student, st-class9-student, urban-street-vendor, young-entrepreneur-sc.